MAPÚA UNIVERSITY
School of Electrical, Electronics, and Computer Engineering
Department of Computer Engineering

DriveReady PH

An LTO Driver's Exam Reviewer Using Retrieval-Augmented Generation, a Local Open-Source LLM, and Reinforcement Learning

Final Project Documentation
Emerging Technologies in Computer Engineering
Instructor: Engr. Dionis A. Padilla

Team Members

Name	Role	Contact
Jezreel Mari Manjares Hembrador	Project lead, developer, documentation	jezreelhembrador@gmail.com
Andrea Bague	Testing, video presentation	asbague@mymail.mapua.edu.ph
Miguel Jose Malvar	Member	mjamalvar@mymail.mapua.edu.ph

Date of Submission: October 12, 2026

Table of Contents

1. Introduction 3
- 1.1 Overview of the Project 3
- 1.2 Objectives and Goals 3
- 1.3 Selected Topic 3
- 1.4 Scope and Limitations 4

2. System Architecture 5
- 2.1 Architecture Overview 5
- 2.2 Offline Indexing Pipeline 6
- 2.3 Question-Answering Flow (RAG) 6
- 2.4 Adaptive Quiz Flow (Reinforcement Learning) 7
- 2.5 Technology Stack 8

3. Implementation 9
- 3.1 Step 1 — Acquiring the Source Documents 9
- 3.2 Step 2 — Cleaning and Chunking 9
- 3.3 Step 3 — Hybrid Retrieval with Reciprocal Rank Fusion 9
- 3.4 Step 4 — Grounded Prompt and Generation 10
- 3.5 Step 5 — Citation Validation and Offline Fallback 11
- 3.6 Step 6 — Question Bank and LLM-Generated Questions 12
- 3.7 Step 7 — Reinforcement-Learning Topic Selection 13
- 3.8 Step 8 — Persistence and User Interface 13
- 3.9 Step 9 — Testing, Repository, and Reproducibility 16

4. Integration with Open-Source LLMs and RAG 17
- 4.1 How the Open-Source Models Are Used 17
- 4.2 Configuration and Setup 17
- 4.3 Challenges and Solutions 18

5. Testing and Evaluation 19
- 5.1 Testing Methodology 19
- 5.2 Retrieval Results 19
- 5.3 Answer Quality: RAG vs. LLM Only 20
- 5.4 LLM-Generated Quiz Questions 20
- 5.5 Reinforcement-Learning Evaluation (Simulation) 20
- 5.6 System Test Cases 21
- 5.7 Summary of Objectives 22

6. Conclusion 23
- 6.1 Summary 23
- 6.2 Key Takeaways and Lessons Learned 23
- 6.3 Future Work and Improvements 23

7. References 24

8. Appendices 25
- Appendix A. Repository Structure and How to Run 25
- Appendix B. Code Listings 25
- Appendix C. Data Samples 32
- Appendix D. Evaluation Questions 32
- Appendix E. Use of AI Tools 35

1. Introduction

1.1 Overview of the Project

Every Filipino who wants a driver's license must pass the Land Transportation Office (LTO) theoretical examination. The official study material is the *Filipino Driver's Manual* (FDM, 2nd Edition), published in two volumes with a combined 293 pages that cover licensing, road signs, pavement markings, right-of-way rules, speed limits, special traffic laws, and fines and penalties [1], [2]. Reading such a long document is hard for first-time applicants, and general-purpose chatbots are risky study partners: they answer fluently but can invent fines, speed limits, or law numbers that are not in the manual, a failure known as *hallucination* [3].

DriveReady PH is an AI study assistant that answers questions about Philippine driving rules **only from the official manual** and shows the page each answer came from. It combines three emerging technologies:

- **Retrieval-Augmented Generation (RAG)** [4], [14]: the question is first used to search the manual; the best passages are then handed to the language model as its only source of facts.
- **A local open-source Large Language Model (LLM)**: Meta Llama 3.1 8B [5] served by Ollama [6] runs on the student's own computer, so the app works offline, costs nothing per question, and keeps conversations private.
- **Reinforcement Learning (RL)**: an adaptive quiz uses a multi-armed bandit with Thompson sampling [7] to learn which topics a student keeps getting wrong and to ask about those topics more often.

The result is a Streamlit web application with four tabs (Ask, Quiz, My Progress, About), a reproducible evaluation suite, and an open-source code repository.

1.2 Objectives and Goals

General objective. To design, implement, and evaluate a locally run RAG system with an open-source LLM and an RL-based adaptive quiz that helps students review for the LTO driver's license examination using only official LTO material.

Specific objectives. The project aims to:

1. **(O1)** build a searchable knowledge base from both volumes of the Filipino Driver's Manual in which every passage keeps its document and page number for citation;
2. **(O2)** implement hybrid retrieval (BM25 keyword search plus embedding search, fused with Reciprocal Rank Fusion) that finds a correct manual page within the top five results (Recall@5) for at least 85% of held-out exam questions;
3. **(O3)** generate answers with a local open-source LLM that are more accurate than the same LLM without retrieval, cite their sources in at least 90% of answers, and refuse out-of-scope questions;
4. **(O4)** implement an adaptive quiz whose topic selection is learned with reinforcement learning, and show in simulation that it helps learners improve faster than random topic selection;
5. **(O5)** run fully offline on a consumer PC, with an offline fallback so that the application never fails during use; and
6. **(O6)** deliver well-commented, tested, open-source code with a README that lets another student install and run the project.

1.3 Selected Topic

The suggested topics for this course were Medical Health Records, a Specialization Course adviser for the Mapúa Computer Engineering Department, and a Recipe Recommender. All three are **domain-specific**

knowledge assistants: a fixed, trusted document collection plus an LLM that must answer from it. DriveReady PH applies the same pattern to **road-safety education**. The domain was chosen because:

- **The source documents are official, public, and authoritative.** The LTO publishes the manual for every license applicant, so answers can be checked page by page.
- **Mistakes have real consequences.** An invented speed limit or fine could mislead a new driver, which makes grounding and citations essential and gives RAG a clear purpose.
- **The manual contains its own exam.** Chapter VI of Volume 1 is an official reviewer with about 230 questions and answers, which provides a ready-made, objective evaluation set and quiz bank.
- **Road safety is a national concern.** The LTO's Road Safety Action Plan names "qualified drivers", meaning drivers who pass the theoretical and practical examination, as its first pillar [1].

1.4 Scope and Limitations

- The knowledge base covers the two volumes of the FDM 2nd Edition (2021–2023). Laws or fines changed after publication are not reflected.
- Chapter VI "Reviewer" (Volume 1, PDF pages 163–200) is deliberately **held out** of the search index; it is used only as the quiz bank and the evaluation set.
- Road-sign images in the manual are not read by the system (text only). Sign questions are answered from the written descriptions.
- The adaptive quiz was evaluated with simulated learners, not with a long-term study of real students.
- DriveReady PH is a study aid, not legal advice; the official LTO materials always prevail.

2. System Architecture

2.1 Architecture Overview

DriveReady PH follows a **three-tier architecture** (Figure 1). The **presentation tier** is a Streamlit web application that runs in the browser. The **application tier** is the `driveready` Python package, which contains the retrieval, RAG, quiz, and reinforcement-learning logic. The **data tier** consists of plain files and a SQLite database: the chunked manual, the embedding matrices, the question bank, and the learner's history. The language model and the embedding model run in a separate local service, **Ollama**, reached through its REST API on `localhost:11434`. Nothing is sent to the internet.

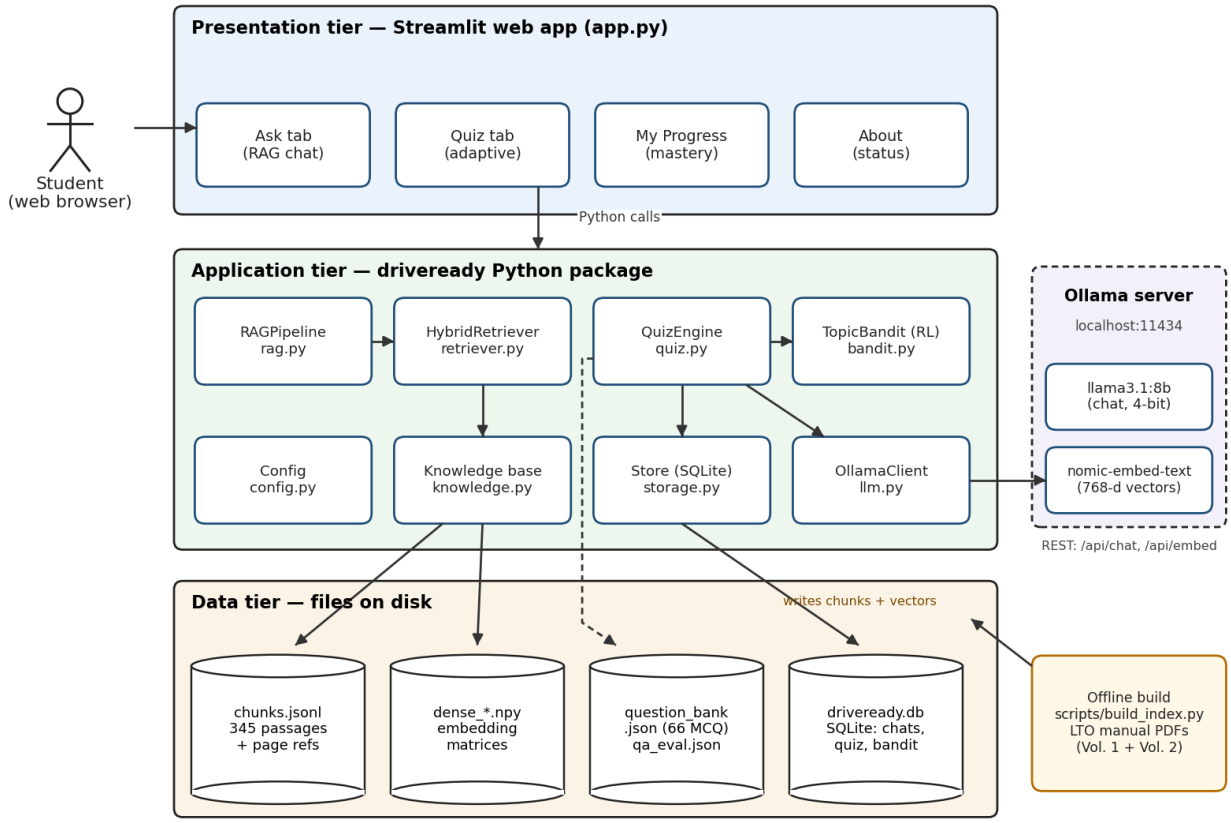


Figure 1. Three-tier architecture of DriveReady PH with the local Ollama model server.

Table 1. Main components and their responsibilities

Tier	Component (file)	Responsibility
Presentation	Streamlit app (`app.py`)	Ask, Quiz, My Progress, and About tabs; shows answers with citations and the bandit's mastery estimates
Application	RAGPipeline (`rag.py`)	Builds the grounded prompt, calls the LLM, validates citations, falls back to extractive mode when the LLM is offline
Application	HybridRetriever (`retriever.py`)	BM25 keyword search, dense cosine search, and Reciprocal Rank Fusion
Application	Knowledge base (`knowledge.py`)	Builds and caches embedding matrices; chooses Ollama (nomic) or LSA embeddings
Application	QuizEngine (`quiz.py`)	Serves questions from the curated bank or asks the LLM to write a new one from a retrieved passage

Tier	Component (file)	Responsibility
Application	TopicBandit (`bandit.py`)	Reinforcement-learning agent (Thompson sampling) that chooses the next quiz topic
Application	OllamaClient (`llm.py`)	REST client for `/api/chat`, `/api/embed`, and `/api/tags` (health check)
Application	Store (`storage.py`)	SQLite access: chat log, quiz attempts, bandit state, feedback
Data	`chunks.jsonl`, `dense_*.npy`	345 manual passages with page numbers and their embedding vectors
Data	`question_bank.json`, `qa_eval.json`	66 multiple-choice questions and 53 evaluation questions from the official reviewer
Data	`driveready.db`	Per-user learning history and bandit parameters
External	Ollama server	`llama3.1:8b` for generation and `nomic-embed-text` for embeddings

2.2 Offline Indexing Pipeline

Before the app can answer questions, the manual is converted into a searchable index by `scripts/build_index.py` (Figure 2). Text is extracted from both PDFs with pypdf, cleaned (hyphenated line breaks are joined and page-number lines removed), and split into overlapping chunks of about 180 words with a 40-word overlap so that a rule is not cut in half at a chunk boundary. Every chunk keeps its document, page number, and nearest heading. The reviewer chapter is removed before chunking. The run produced **345 chunks** from 246 pages that contain text (mean 124 words per chunk). Two indexes are then built: a BM25 index over the chunk words and a dense matrix of embedding vectors (768-dimensional `nomic-embed-text` vectors from Ollama, or 100-dimensional Latent Semantic Analysis (LSA) vectors computed from the manual with NumPy when Ollama is not available).

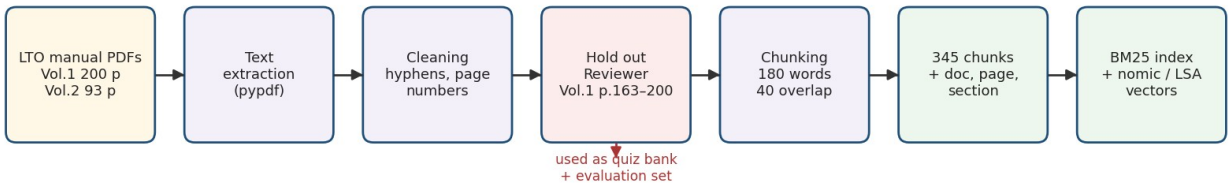


Figure 2. Offline indexing pipeline: from the LTO PDFs to the BM25 and vector indexes.

2.3 Question-Answering Flow (RAG)

Figure 3 shows what happens when a student asks a question. The question is searched in two ways at once. BM25 is strong on exact words such as "fire hydrant" or "R.A. No. 10913", while embedding search matches meaning even when the student uses different words. The two ranked lists are merged with Reciprocal Rank Fusion, and the five best passages are numbered [1]–[5] and placed in the prompt. The LLM must answer only from them and cite them. A validator removes citation numbers that do not exist and flags answers that cite nothing. If Ollama is not running, the system skips generation and quotes the best passage, so the app never stops working.

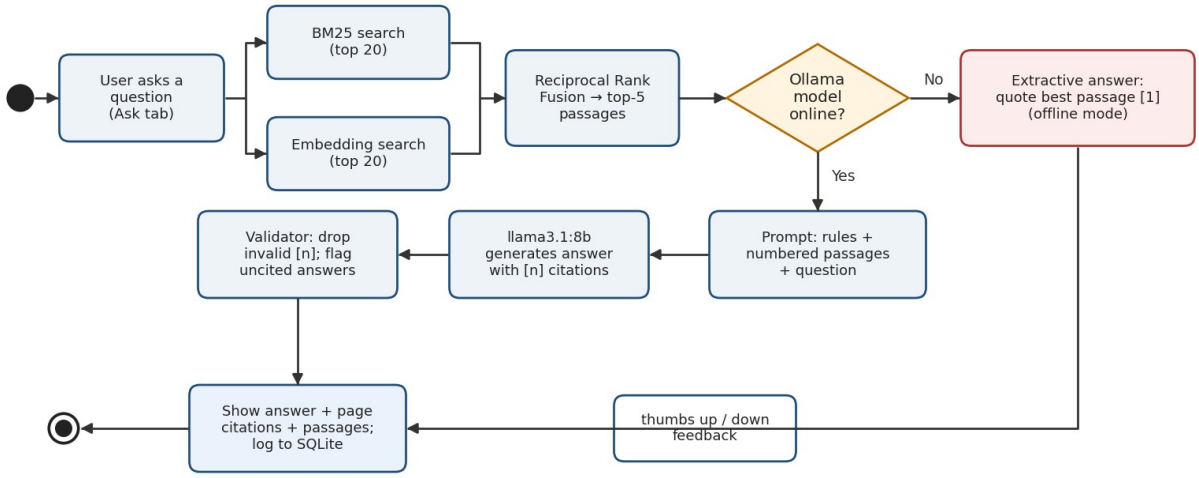


Figure 3. Flowchart of the retrieval-augmented question-answering process.

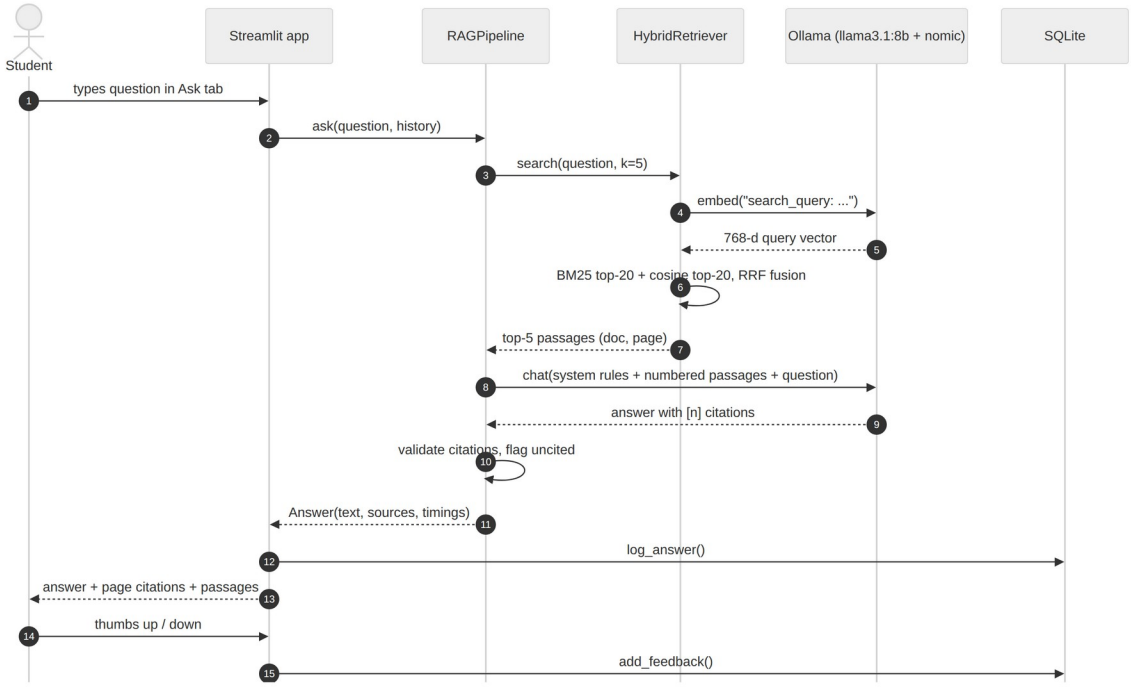


Figure 4. Sequence diagram of one question in the Ask tab.

2.4 Adaptive Quiz Flow (Reinforcement Learning)

The quiz is a reinforcement-learning loop (Figure 5) framed in the standard agent–environment terms of RL [13]. The **agent** is the TopicBandit; its **actions** are the eight quiz topics; the **environment** is the student; and the **reward** is 1 when the student answers wrongly, meaning the agent found a weak spot worth practising. For each question, the agent samples an error rate for every topic from its Beta belief and chooses the topic with the highest sample. The question comes from the LLM, written from a retrieved manual passage, or from the curated bank. After the answer, the belief of that topic is updated and saved, and the My Progress tab displays the resulting mastery estimates.

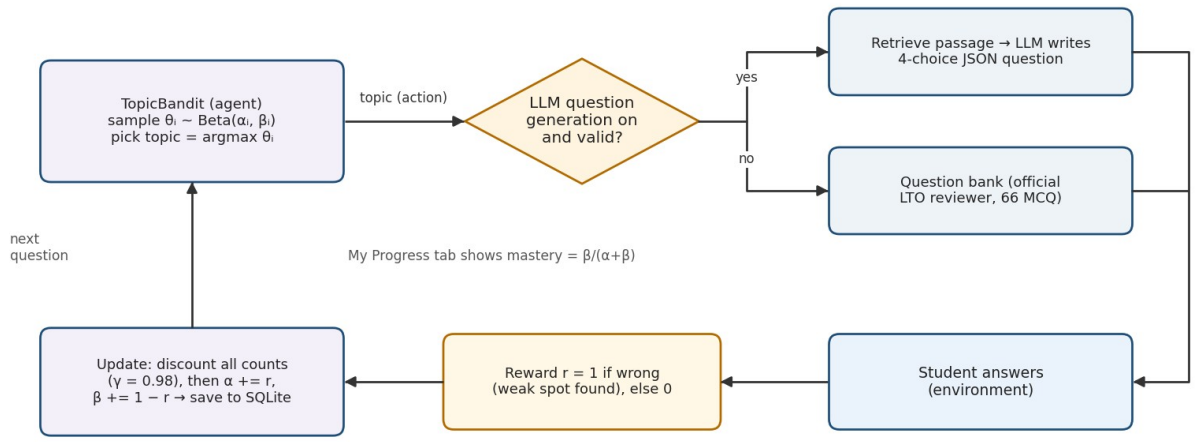


Figure 5. Reinforcement-learning loop of the adaptive quiz.

2.5 Technology Stack

Table 2. Technologies used

Layer	Technology	Why it was chosen
Language	Python 3.10–3.14 (tested on 3.14.6, Windows 11)	Standard for AI; all libraries available on Windows
LLM runtime	Ollama 0.4+	One-command install of open-source models; local REST API; runs on CPU or GPU
Generation model	Llama 3.1 8B Instruct (Q4 quantized, 4.9 GB)	Strong open model that follows instructions; fits in 16–32 GB RAM
Embedding model	nomic-embed-text (137M params, 768-d)	Open-source (Apache 2.0) retrieval embeddings served by the same Ollama runtime [8]
Fallback embeddings	LSA: TF-IDF + truncated SVD in NumPy	Works with no downloads or extra packages (course Lesson 4: vector semantics)
Ablation embeddings	gensim FastText (optional)	Character n-grams handle unseen words [12]; compared in the evaluation
Keyword search	rank-bm25 (Okapi BM25)	Classic term-weighting retrieval [9] (course Lesson 1)
PDF parsing	pypdf	Pure Python, permissive licence
User interface	Streamlit	Fast web UI in pure Python, with chat components [15]
Storage	SQLite (Python standard library)	Zero-configuration persistence
Quality	pytest, Ruff, GitHub Actions	Unit tests, linting, continuous integration

3. Implementation

This section explains the implementation step by step, in the order in which the system was built. Each step names the file and shows the key code.

3.1 Step 1 — Acquiring the Source Documents

The two official PDFs, *Filipino Driver's Manual* Volume 1 (200 pages) and Volume 2 (93 pages), are downloaded from the LTO website by `scripts/download_docs.py` into `data/raw/`. Their URLs and file names are stored in `config.SOURCES`, together with the list of held-out reviewer pages. The PDFs are not committed to the repository; only the derived text chunks are.

3.2 Step 2 — Cleaning and Chunking

`ingest.py` extracts each page, cleans it, and splits it into overlapping windows of words. Overlap matters because a rule that starts at the end of one chunk is still complete in the next one. The page number travels with every chunk, so the citation shown to the user is exact.

Listing 1. Overlapping word windows (driveready/ingest.py)

```
def split_words(text: str, size: int, overlap: int) -> list[str]:
    words = text.split()
    if len(words) <= size:
        return [" ".join(words)] if words else []
    step = max(1, size - overlap)
    pieces = []
    for start in range(0, len(words), step):
        piece = words[start:start + size]
        pieces.append(" ".join(piece))
        if start + size >= len(words):
            break
    return pieces
```

One stored chunk looks like this (`data/processed/chunks.jsonl`):

```
{
  "chunk_id": "FDM-Vol1-p115-c1",
  "doc_id": "FDM-Vol1",
  "page": 115,
  "section": "Driving On The Road",
  "text": "Before Starting and Driving on the Road DRIVING ON THE ROAD TRAFFIC LIGHTS AND
SIGNALS RED SIGNAL means you must bring your vehicle to a STOP at a marked line. FLASHING RED
SIGNAL means bring your vehicle to a STOP and proceed onl ..."
}
```

3.3 Step 3 — Hybrid Retrieval with Reciprocal Rank Fusion

The tokenizer for BM25 lowercases text, removes common English and Filipino stop words, and keeps tokens such as `ra-10913` or `60kph` intact, which is a direct application of the text-processing techniques from Lesson 1. The dense retriever compares L2-normalized vectors with cosine similarity, which is the word-vector similarity from Lesson 4. The two ranked lists are fused with Reciprocal Rank Fusion (RRF) [10]: each passage receives the score $\Sigma\ 1/(k + \text{rank})$, with $k = 60$. RRF needs no tuning and rewards passages that both retrievers rank highly.

Listing 2. Hybrid search with Reciprocal Rank Fusion (driveready/retriever.py)

```
    def search(self, query: str, k: int = config.TOP_K, mode: str = "hybrid") -> list[Hit]:
        bm = self.bm25_search(query) if mode in ("hybrid", "bm25") else []
        de = self.dense_search(query) if mode in ("hybrid", "dense") else []
        scores: dict[int, float] = {}
        bm_rank = {idx: r for r, (idx, _) in enumerate(bm, start=1)}
        de_rank = {idx: r for r, (idx, _) in enumerate(de, start=1)}
        for idx, r in bm_rank.items():
            scores[idx] = scores.get(idx, 0.0) + 1.0 / (config.RRF_K + r)
        for idx, r in de_rank.items():
            scores[idx] = scores.get(idx, 0.0) + 1.0 / (config.RRF_K + r)
        order = sorted(scores, key=scores.get, reverse=True)[:k]
        return [Hit(self.chunks[i], scores[i], bm_rank.get(i), de_rank.get(i)) for i in order]
```

3.4 Step 4 — Grounded Prompt and Generation

The prompt is the main safety mechanism of the system. The system message forbids outside knowledge, requires a citation after each fact, and defines an exact refusal sentence. The five retrieved passages are inserted with their number, document, and page, followed by the question. The last four chat turns are included so that follow-up questions such as "what about for trucks?" make sense.

Listing 3. System prompt (driveready/rag.py)

```
SYSTEM_PROMPT = f"""You are DriveReady PH, a friendly reviewer for the Philippine LTO driver's
license exam.
Rules:
1. Answer ONLY with facts from the numbered manual passages provided. Never use outside
knowledge.
2. After each fact, cite its passage number in square brackets, e.g. [2].
3. If the passages do not contain the answer, reply exactly: "{NOT_FOUND}"
4. Keep the answer short: 2 to 5 sentences, in simple language.
   Reply in the language named in the "Answer language" line of the user message.
5. Do not invent fines, speeds, or law numbers that are not written in the passages."""
```

The request is sent to Ollama's `/api/chat` endpoint with a low temperature (0.1) for consistent, factual answers and a 4,096-token context window, which comfortably fits the five passages:

Listing 4. Calling the local LLM through Ollama (driveready/llm.py)

```
    def chat(self, messages: list[dict], temperature: float = config.LLM_TEMPERATURE,
             json_mode: bool = False) -> ChatResult:
        payload = {
            "model": self.model,
            "messages": messages,
            "stream": False,
            "options": {"temperature": temperature, "num_ctx": 4096},
        }
        if json_mode:
            payload["format"] = "json"   # Ollama constrains the output to valid JSON
        t0 = time.perf_counter()
        try:
            r = requests.post(f"{self.url}/api/chat", json=payload, timeout=self.timeout)
        except requests.RequestException as e:
            raise OllamaError(f"Could not reach Ollama: {e}") from e
        if r.status_code == 404:
            raise OllamaError(f"Model '{self.model}' not found. Run: ollama pull
{self.model}")
        r.raise_for_status()
        text = r.json()["message"]["content"]
        return ChatResult(text=text, seconds=time.perf_counter() - t0, model=self.model)
```

3.5 Step 5 — Citation Validation and Offline Fallback

LLMs sometimes cite a passage number that does not exist (for example [7] when only five passages were given). The validator keeps only valid citation numbers, maps them back to the manual pages, and marks the answer as *ungrounded* if it cites nothing. The app then warns the user to double-check. When Ollama is not running, RAG`Pipeline.ask` returns an extractive answer that quotes the most relevant sentences of the best passage.

Listing 5. Extracting valid citations (driveready/rag.py)

```
def extract_citations(text: str, n_passages: int) -> list[int]:
    found: list[int] = []
    for group in _CITE.findall(text):
        for num in group.split(","):
            k = int(num)
            if 1 <= k <= n_passages and k not in found:
                found.append(k)
    return found
```

Listing 6. The complete RAG pipeline (driveready/rag.py)

```
    def ask(self, question: str, history: list[dict] | None = None,
            mode: str = "hybrid", use_llm: bool = True) -> Answer:
        t0 = time.perf_counter()
        hits = self.retriever.search(question, k=self.top_k, mode=mode)
        t_ret = time.perf_counter() - t0
        if not use_llm or not self.client.available():
            text = extractive_answer(question, hits)
            return Answer(question, text, hits, cited=[1] if hits else [], mode="extractive",
                          timings={"retrieval_s": t_ret, "llm_s": 0.0,
                                   "total_s": time.perf_counter() - t0})
        try:
            res = self.client.chat(build_messages(question, hits, history))
        except OllamaError as e:
            text = extractive_answer(question, hits) + f"\n\n_LLM error: {e}_"
            return Answer(question, text, hits, cited=[1] if hits else [], mode="extractive",
                          timings={"retrieval_s": t_ret, "llm_s": 0.0,
                                   "total_s": time.perf_counter() - t0})
        text = strip_invalid_citations(res.text.strip(), len(hits))
        cited = extract_citations(text, len(hits))
        grounded = bool(cited) or NOT_FOUND.lower()[:30] in text.lower()
        return Answer(question, text, hits, cited=cited, mode="llm", grounded=grounded,
                      timings={"retrieval_s": t_ret, "llm_s": res.seconds,
                               "total_s": time.perf_counter() - t0},
                      model=res.model)
```

3.6 Step 6 — Question Bank and LLM-Generated Questions

The **question bank** contains 66 multiple-choice questions, eight or nine per topic, taken from the official reviewer chapter. The correct answer is the official LTO answer, and three plausible wrong choices were written by the team. When the student turns on *Let the LLM write new questions*, `QuizEngine.generate` retrieves a passage for the chosen topic and asks the LLM, in Ollama's JSON mode, for a new four-choice question. The output is accepted only if it passes `validate_question`; otherwise the engine silently falls back to the bank.

Listing 7. Prompt for generating quiz questions (driveready/quiz.py)

```
GEN_PROMPT = """You write multiple-choice review questions for the Philippine LTO driver's
license exam.
Use ONLY the manual passage below. Write ONE question about a fact stated in the passage.
Return JSON with keys: "question" (string), "choices" (list of exactly 4 short strings),
"answer" (index 0-3 of the correct choice), "explanation" (one sentence quoting the fact).
The three wrong choices must be plausible but clearly wrong according to the passage.

Topic: {topic}
Passage ({source}):
{text}"""
```

Listing 8. Validating generated questions (driveready/quiz.py)

```
def validate_question(d: dict) -> bool:
    try:
        choices = [str(c).strip() for c in d["choices"]]
        ans = int(d["answer"])
        return (len(str(d["question"]).strip()) > 10 and len(choices) == 4
                and len(set(c.lower() for c in choices)) == 4 and all(choices)
                and 0 <= ans < 4 and len(str(d.get("explanation", "")).strip()) > 0)
    except (KeyError, TypeError, ValueError):
        return False
```

3.7 Step 7 — Reinforcement-Learning Topic Selection

Each topic *i* keeps a Beta(α*i*, β*i*) distribution that represents the agent's belief about the student's error rate on that topic; both start at 1 (uniform). **Thompson sampling** [7], [11] draws one sample per topic and selects the largest, so topics with many recorded mistakes are usually chosen (exploitation), while topics with few answers are still chosen sometimes because their samples vary widely (exploration). Because students improve, all counts are discounted toward the prior by γ = 0.98 before every update, so recent answers weigh more than old ones.

Listing 9. Thompson sampling and the discounted update (driveready/bandit.py)

```
    def select(self) -> int:
        samples = self.rng.beta(self.alpha, self.beta)
        return int(np.argmax(samples))

    def update(self, arm: int, correct: bool) -> None:
        a0, b0 = self.prior
        # discount: shrink all evidence toward the prior so recent answers count more
        self.alpha = a0 + self.gamma * (self.alpha - a0)
        self.beta = b0 + self.gamma * (self.beta - b0)
        if correct:
            self.beta[arm] += 1.0
        else:
            self.alpha[arm] += 1.0
        self.pulls[arm] += 1
```

The estimated mastery of a topic, shown in the My Progress tab, is the posterior probability of a correct answer, β/(α + β).

3.8 Step 8 — Persistence and User Interface

`storage.py` stores every question, quiz attempt, bandit state, and thumbs-up/down rating in a local SQLite database, so progress survives restarting the app. `app.py` builds the Streamlit interface. Heavy objects (the index, the pipelines) are created once with `st.cache_resource`. The sidebar shows whether the LLM is online, and each answer shows its citations, the retrieved passages with their BM25 and dense ranks, and the response time.

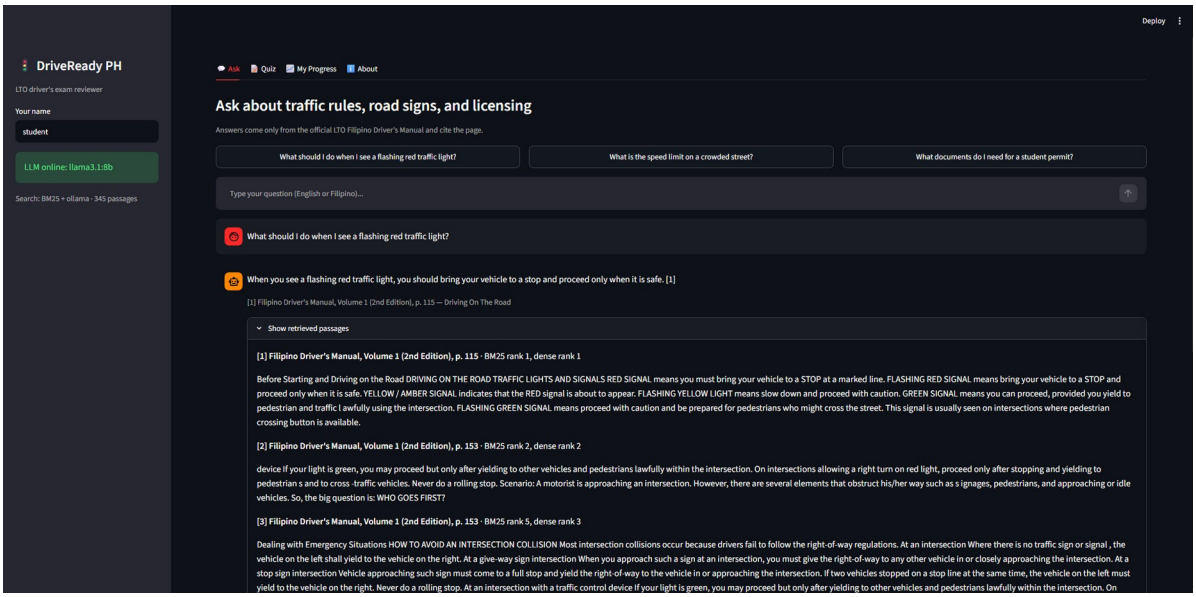


Figure 6. Ask tab: a grounded answer with page citations.

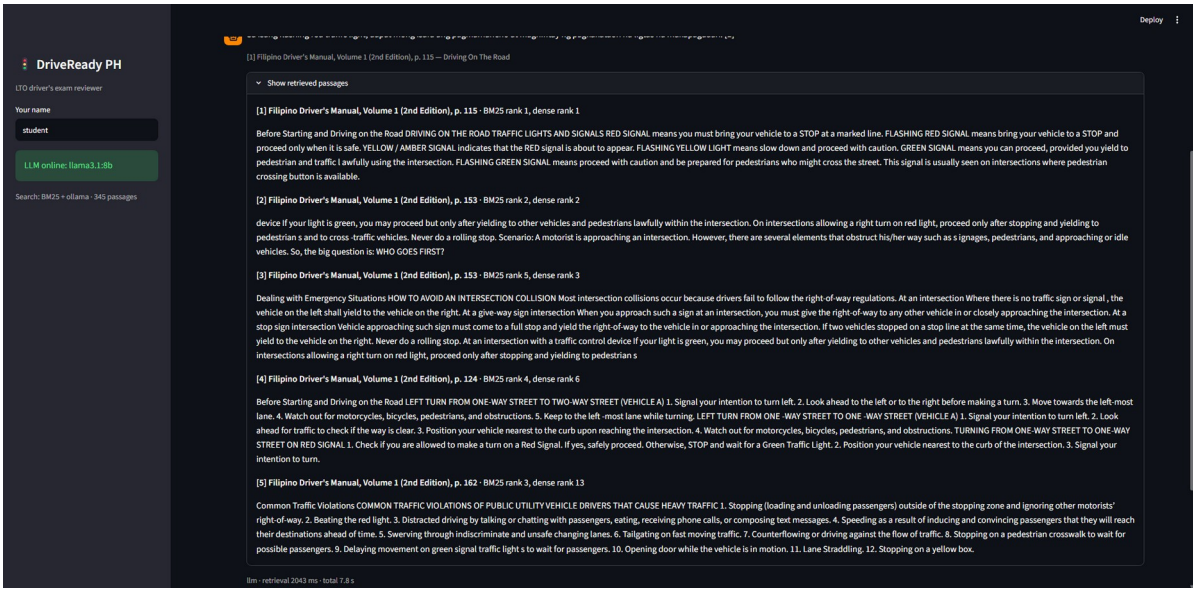


Figure 7. Ask tab: the five retrieved passages with their BM25 and dense ranks.

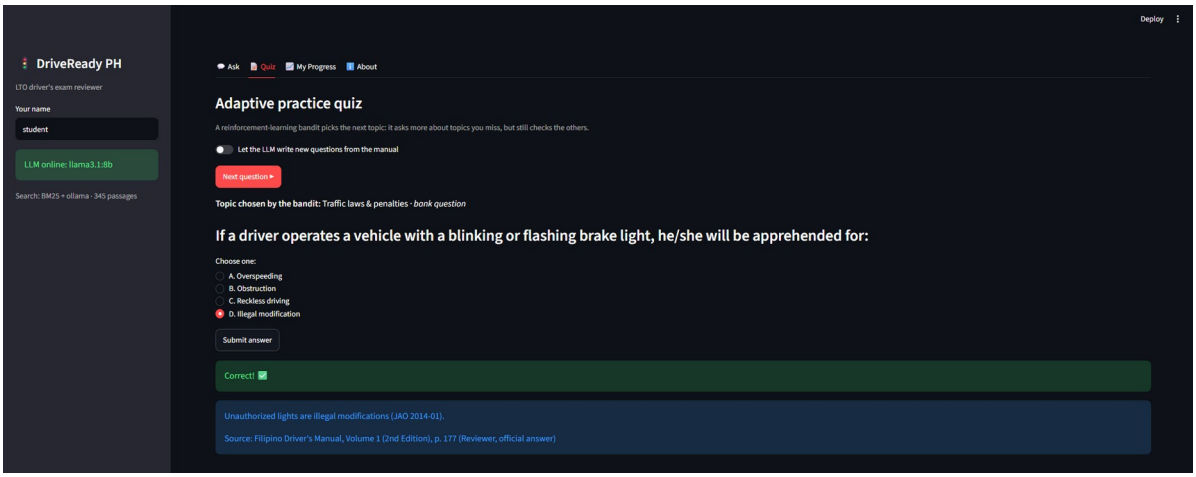


Figure 8. Quiz tab: the topic is chosen by the RL bandit.

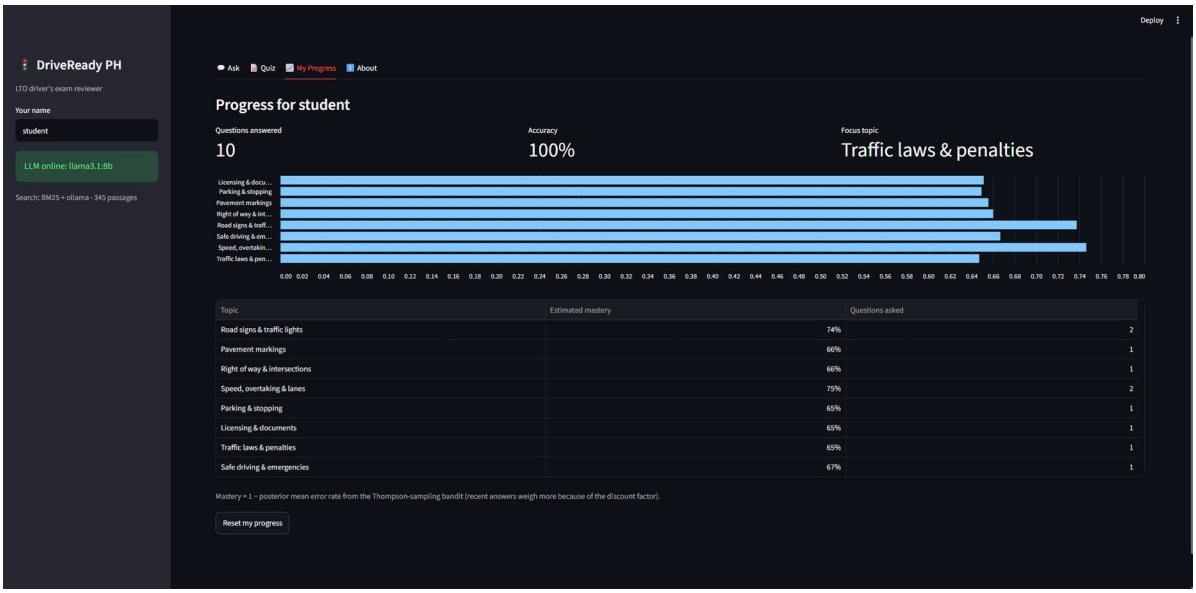


Figure 9. My Progress tab: per-topic mastery estimated by the bandit.

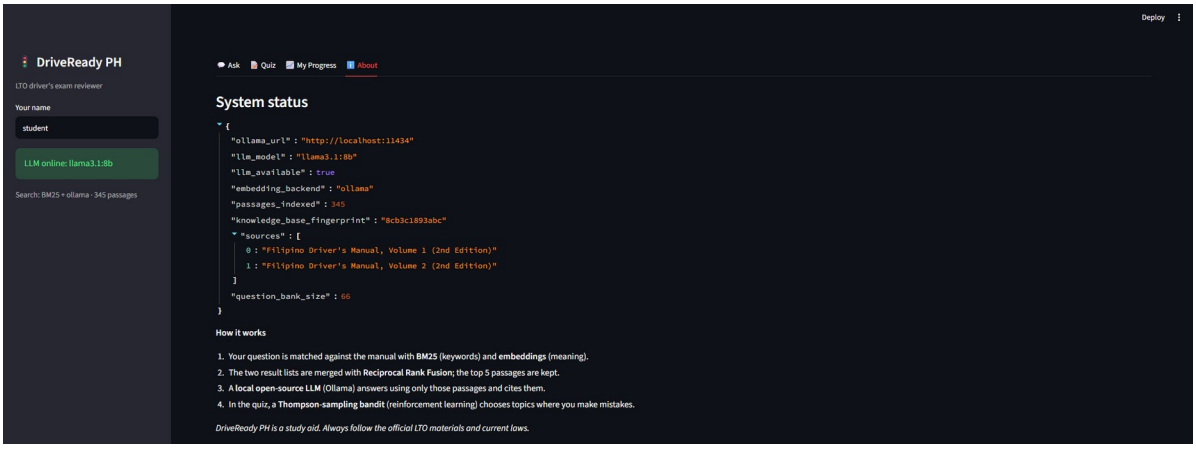


Figure 10. About tab: system status and knowledge-base fingerprint.

3.9 Step 9 — Testing, Repository, and Reproducibility

The repository contains 24 unit tests (`tests/`) that use a small fake manual and a fake LLM client, so they run in under a second without Ollama. Ruff checks code style, and a GitHub Actions workflow runs both on every push. The scripts in `scripts/` regenerate every number in this report and write it as JSON to `reports/`.

4. Integration with Open-Source LLMs and RAG

4.1 How the Open-Source Models Are Used

DriveReady PH uses **two open-source models**, both served locally by Ollama, plus one model trained by the project:

Table 3. Models used in DriveReady PH

Model	Type / size	Licence	Role in the system
Llama 3.1 8B Instruct (`llama3.1:8b`)	Decoder-only LLM, 8B parameters, 4-bit (Q4_K_M), 4.9 GB	Llama 3.1 Community License	Writes cited answers in the Ask tab; writes new quiz questions in JSON mode
nomic-embed-text	Text embedding model, 137M parameters, 768-d output, 274 MB	Apache 2.0	Encodes manual passages and questions for dense retrieval
LSA (built in)	TF-IDF matrix reduced to 100 dimensions with SVD (NumPy)	Project code (MIT)	Offline fallback for dense retrieval
FastText (gensim, optional)	Skip-gram with character n-grams, 100-d, trained on the manual	LGPL (gensim)	Ablation baseline in the evaluation

The **RAG** design keeps the LLM's job narrow: it does not need to *know* traffic law, only to read five passages and restate the relevant facts with citations. This is why a mid-size 8B model, which would hallucinate on its own, can be trusted here. Retrieval also makes the system **updatable**: if the LTO publishes a 3rd edition, re-running `build_index.py` updates the knowledge without retraining any model.

4.2 Configuration and Setup

Setup takes five commands on Windows, macOS, or Linux (the full guide is in the README, Appendix A):

Listing 10. Installation and start-up

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
ollama pull llama3.1:8b
ollama pull nomic-embed-text
python scripts/build_index.py
streamlit run app.py
```

Table 4. RAG and LLM configuration (driveready/config.py)

Parameter	Value	Reason
Chunk size / overlap	180 words / 40 words	One rule or table per chunk; overlap avoids splitting rules
Candidates per retriever	20	Enough recall before fusion
Passages given to the LLM (k)	5	Fits the context window; more passages add noise
RRF constant	k = 60	Standard value from Cormack et al. [10]
LLM temperature	0.1 (answers), 0.7 (quiz generation)	Factual answers; varied quiz questions
Context window (num_ctx)	4,096 tokens	System prompt + 5 passages + 4 chat turns
Embedding prefixes	`search_document:` / `search_query:`	Required by nomic-embed-text for asymmetric retrieval [8]
Bandit prior / discount	Beta(1, 1) / γ = 0.98	Uninformative start; recent answers weigh more

Parameter	Value	Reason
Environment overrides	`DRIVEREADY_LLM`, `DRIVEREADY_EMBED`, `DRIVEREADY_DENSE`, `OLLAMA_URL`	Swap models (e.g., `llama3.2:3b` for 8 GB PCs) without code changes

Hardware used for the evaluation. The evaluation reported in Section 5 was run on: Windows-11-10.0.26200-SP0, Python 3.14.6. The development PC has 32 GB of RAM. Ollama loads the 4.9 GB model into memory on the first request (which therefore takes longer) and keeps it loaded for later questions.

4.3 Challenges and Solutions

Table 5. Challenges encountered and how they were solved

Challenge	Solution
Evaluation leakage. The manual contains its own exam reviewer. If it were indexed, the system would retrieve the answer key itself and look perfect.	The reviewer chapter (Vol. 1, PDF pp. 163–200) is removed before indexing and used only as the quiz bank and evaluation set. Each evaluation question was mapped to the content page that teaches the answer.
Hallucinated or wrong citations. LLMs sometimes cite passages that do not exist or answer from memory.	Strict system prompt with an exact refusal sentence; a citation validator that drops invalid numbers and flags uncited answers; an LLM-only baseline to measure the improvement.
Exact terms vs. paraphrases. Students ask "Can I use my phone at a red light?", while the manual says "Anti-Distracted Driving Act".	Hybrid retrieval: BM25 catches exact terms and numbers; embeddings catch paraphrases; RRF merges both lists.
Demo reliability. If Ollama is closed or a model is missing, an LLM app usually crashes.	Health check before every request and an extractive offline mode that still returns the best manual passage with its page.
Invalid quiz JSON from the LLM. Generated questions sometimes had three choices or a bad answer index.	Ollama JSON mode, a strict validator (4 distinct choices, valid index, explanation), and automatic fallback to the curated bank.
Inconsistent source material. On uphill parking with a curb, the content chapter (p. 129) says to turn the wheels "into the curb", while the reviewer answer (p. 188) says "away from the curb".	Such questions were left out of the evaluation set; the app always cites the page so students can see which part of the manual an answer came from.
PDF artefacts. Many pages are mostly images (road signs), and the text contains broken hyphenation and page numbers.	Cleaning rules in `ingest.py`; image-only pages produce no chunks; sign meanings are taken from the written descriptions.
Weak dense vectors without internet. FastText or LSA built only from a 46,000-word manual gives lower-quality vectors.	Use the pre-trained open-source `nomic-embed-text` through Ollama as the default (Recall@5 rose from 0.79 with LSA to 0.98); keep LSA only as the offline fallback.
The LLM answered English questions in Filipino. The first full evaluation showed Llama 3.1 switching to Filipino because of the Philippine context and a loose prompt rule. Correct facts then failed the English answer check.	The prompt now states the reply language explicitly (English unless the question is in Filipino, detected from common Tagalog function words), and a unit test checks it. Section 5.3 compares both runs.
Python 3.14 on the team PC. The gensim package has no Windows wheels for Python 3.13+, and PowerShell blocked the virtual-environment activation script.	gensim became optional with a NumPy-only LSA fallback; `setup_windows.bat`, `run_app.bat` and `run_evaluation.bat` call the environment's Python directly, so no script activation is needed.
Showing that RL actually helps. A real multi-week user study was not possible within the project timeline.	A simulation with 500 paired synthetic learners and five policies, including an oracle upper bound; the findings are reported honestly, including the case where RL does not help (Section 5.5).

5. Testing and Evaluation

5.1 Testing Methodology

Testing was done at four levels:

1. **Unit tests** (pytest, 24 tests) verify chunking, tokenization, BM25 and hybrid ranking, citation parsing, the offline fallback, question validation, bandit behaviour (focus on weak topics, exploration, discounting, persistence), and SQLite storage. A fake LLM client makes them deterministic.
2. **Retrieval evaluation** (`scripts/evaluate.py`) measures how often a correct manual page is retrieved for 48 held-out exam questions.
3. **Answer evaluation** compares DriveReady's RAG answers with the same LLM answering *without* the manual (LLM-only baseline) on the same questions, including 5 out-of-scope questions that should be refused.
4. **RL evaluation** (`scripts/simulate_bandit.py`) compares the bandit with baseline policies on 500 simulated learners. **System tests** check the complete application by hand (Section 5.6).

Evaluation dataset

The evaluation set (`data/eval/qa_eval.json`) contains **53 questions**: 48 answerable questions taken or adapted from the official reviewer and from the law sections of the manual (e.g., R.A. No. 4136 [16]), eight topics with five to eight questions each, plus 5 out-of-scope questions (for example, toll fees or today's diesel price) that the manual cannot answer. Each answerable question lists its **gold page(s)**, the content page(s) that teach the answer, and the **required facts** that a correct answer must contain (for example, `20` for the school-zone speed limit). Appendix D lists all questions.

Table 6. Evaluation metrics

Metric	Definition
Recall@k	Share of questions for which at least one gold page appears in the top k retrieved passages
MRR@10	Mean reciprocal rank of the first gold passage (1 = always first)
Answer accuracy	Share of answerable questions whose answer contains every required fact
Citation rate / gold-citation rate	Share of answers that cite at least one passage / that cite a gold page
Correct refusals	Share of out-of-scope questions answered with a refusal instead of an invented answer
Latency	Median (p50) and 95th-percentile (p95) time per question
Quiz validity	Share of LLM-generated questions that pass the validator
Learning gain (RL)	Increase in a simulated learner's average topic mastery after a 60-question session

5.2 Retrieval Results

Table 7. Retrieval results on 48 answerable questions (reports/eval_retrieval.json)

Retriever	Recall@1	Recall@3	Recall@5	MRR@10	Latency p50 (ms)
BM25 only	0.65	0.81	0.85	0.74	0.41
LSA only	0.52	0.71	0.79	0.63	0.11
Hybrid BM25+LSA	0.63	0.77	0.85	0.72	0.67
nomic-embed only	0.77	0.92	0.98	0.85	2084.92
Hybrid BM25+nomic (app)	0.71	0.94	0.96	0.82	2089.66

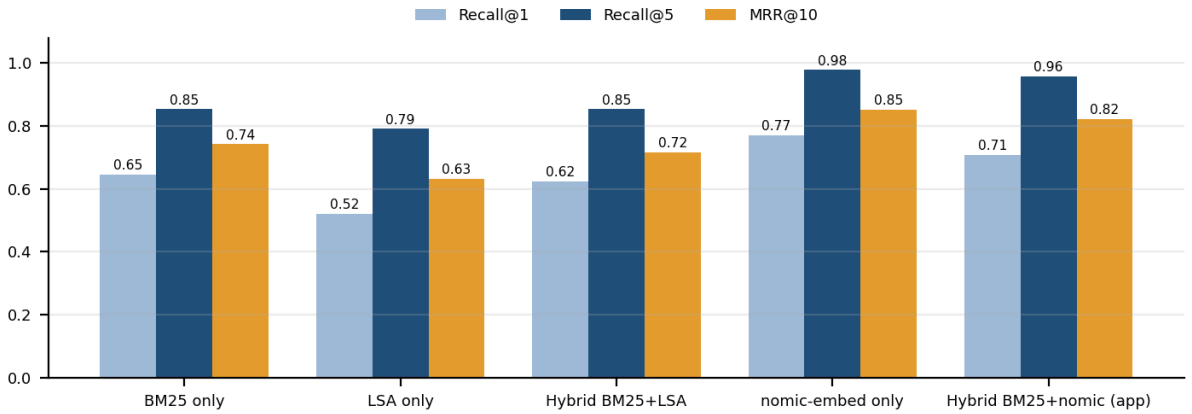


Figure 11. Recall@1, Recall@5, and MRR@10 of each retrieval method.

BM25 alone finds a gold page in the top five for 85.4% of the questions, because LTO exam questions often reuse the manual's own wording ("fire hydrant", "school zone", "double solid yellow"). Vectors built only from the manual are weaker: LSA reaches Recall@5 0.79 and FastText 0.77 (development run, reports/eval_retrieval_dev.json), and fusing LSA with BM25 does not improve on BM25 alone. The pre-trained open-source `nomic-embed-text` model is clearly the best single retriever (Recall@5 **0.98**, MRR 0.85), because it matches meaning rather than exact words. The hybrid BM25 + nomic retriever used by the app reaches Recall@5 **0.96** and Recall@3 0.94, meeting the 0.85 target of objective O2. On this question set the hybrid retriever is slightly behind nomic alone (one question out of 48), because BM25 occasionally pushes a keyword-heavy but wrong page into the top five. Hybrid search was kept in the app because BM25 is more reliable for exact law numbers and amounts, which are common in real questions; making RRF weight the two retrievers differently is listed as future work. Dense search costs about 2.1 s per question, almost all of it for embedding the question with Ollama on the CPU, compared with under 1 ms for BM25. Note that the gold pages were located by searching the manual for the answer facts, which may favour keyword retrieval slightly.

5.3 Answer Quality: RAG vs. LLM Only

Table 8. Answer quality with llama3.1:8b (reports/eval_answers.json)

Metric	DriveReady (RAG)	LLM only (no manual)
Accuracy on answerable questions (n = 48)	93.8%	56.2%
Correct refusals on out-of-scope questions (n = 5)	100.0%	100.0%
Answers that cite a passage	95.8%	not applicable
Answers that cite a gold page	93.8%	not applicable
Latency p50 / p95 (s)	7.58 / 8.73	2.97 / –

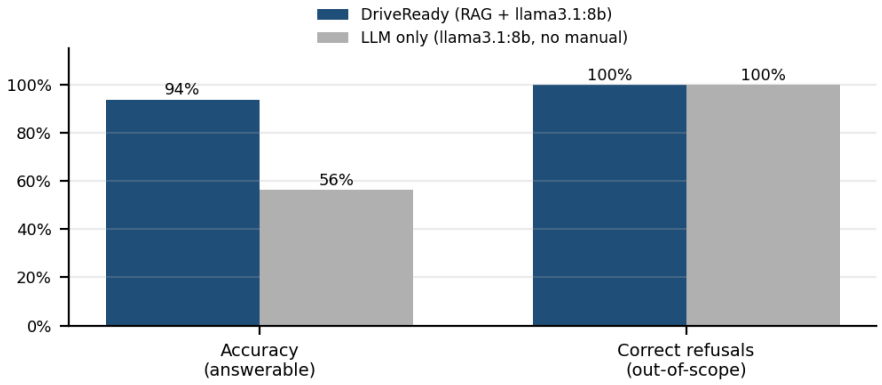


Figure 12. Accuracy and correct refusals: RAG versus the same LLM without the manual.

With retrieval, llama3.1:8b answered **93.8%** of the answerable questions correctly, compared with **56.2%** when the same model answered from its own memory, a gain of +37.6 percentage points. 95.8% of RAG answers cited the manual, and 93.8% cited a gold page, so a student can check almost every answer against the exact page. Both systems declined all five out-of-scope questions, so the advantage of RAG here is accuracy and verifiability rather than refusal. The LLM-only errors were almost all Philippine-specific numbers that a general model has not memorized: it said parking is prohibited within 3 m of a fire hydrant (the manual says 4 m), gave 60 km/h for open country roads (80 km/h), a 6-month student permit (1 year), a ₱20,000–₱50,000 fine for driving without a license (₱3,000), and a TOP valid for 60 days (72 hours). This is exactly the gap that RAG closes. RAG costs time: a median of 7.58 s per answer versus 2.97 s, mostly because the prompt is longer and the question must be embedded first.

Error analysis. RAG missed three questions. (1) School-zone speed: the correct page was retrieved, but the PDF table lost its column layout during text extraction, and the model read 30 km/h from the wrong column instead of 20 km/h. (2) Exceptions to speed limits: the model listed rows of the speed table instead of the emergency-vehicle exemptions on the next page. (3) Blood alcohol limit: the page with the 0.05% BAC table was not in the top five passages, and the model correctly said the passages did not contain the answer instead of guessing. The first two errors point to better table extraction as future work; the third shows the grounding rules working as intended.

Accuracy here is measured with a strict keyword rule (every required fact must appear in the answer). It can count an answer as wrong when the answer is correct but worded differently, so the figures are a conservative estimate. The per-question answers are kept in `reports/eval_answers.json` for manual review.

First evaluation run and the language fix

The first full run (reports/eval_answers_run1.json) gave a surprising result: RAG scored only 52.1% against 58.3% for the LLM alone. Reading the answers showed why. Llama 3.1 answered most English questions **in Filipino**, because the prompt allowed Filipino replies and the passages are about the Philippines. Many of those answers were factually right (for example, "4 metros mula sa isang fire hydrant" or "Php 20,000.00 hanggang Php 80,000.00"), but they failed the English keyword check. The scorer also did not count "I don't know" as a refusal. Both problems were fixed: the prompt now names the reply language explicitly, and the refusal check recognizes common "I don't know" phrasings. The table above is from the second run with the fix; the comparison shows how sensitive an LLM system is to small prompt details, and why reading individual outputs matters as much as the summary numbers.

Table 9. First run versus second run (after the language fix)

Metric	Run 1: RAG	Run 1: LLM only	Run 2: RAG	Run 2: LLM only
Accuracy (answerable)	52.1%	58.3%	93.8%	56.2%
Correct refusals	100.0%	0.0%	100.0%	100.0%
Cites a gold page	91.7%	–	93.8%	–

5.4 LLM-Generated Quiz Questions

The LLM generated 16 questions (2 per topic). **16 of 16 (100.0%)** passed the validator, with a median generation time of 8.35 s. Invalid outputs never reach the student, because the engine falls back to the curated bank. A sample generated question:

```
Topic: Road signs & traffic lights
Q: What type of sign is used to warn of a railway level crossing?
  A. Regulatory Sign
  B. Warning Sign
  C. Guide / Informative Sign   <- correct
  D. Traffic Instruction Sign
Explanation: Warning Signs are used to warn of railway level crossings, among other hazards.
Source: Filipino Driver's Manual, Volume 1 (2nd Edition), p. 65
```

5.5 Reinforcement-Learning Evaluation (Simulation)

Each simulated learner starts with a hidden probability of answering correctly for each of the 8 topics, learns after every question (a large gain after a wrong answer, because the explanation is read, and a small gain after a correct one), and takes a 60-question session. Two learner types were tested: **A, mixed skills** (every topic between 30% and 90%) and **B, two weak topics** (two topics at 25–45%, six at 75–95%), which is the typical reviewer who is good overall but has blind spots. Every policy faced the same 500 learners (a paired design).

Table 10. RL simulation results, 500 learners × 60 questions (reports/bandit_simulation.json)

Policy	A: final mastery	A: gain vs random	B: final mastery	B: gain vs random	B: weakest topic	B: time on 2 weakest
Random	0.773	+0%	0.838	+0%	0.617	24.9%
Round-robin	0.777	+2.2%	0.842	+2.8%	0.647	24.9%
Epsilon-greedy (ε=0.2)	0.755	-10.7%	0.847	+7.4%	0.629	63.8%
Thompson (γ=1.0)	0.774	+0.4%	0.855	+14.7%	0.693	46.5%
Thompson (γ=0.98, app)	0.776	+1.5%	0.854	+13.6%	0.688	41.2%
Oracle (upper bound)	0.797	+14.6%	0.874	+32.3%	0.854	70.6%

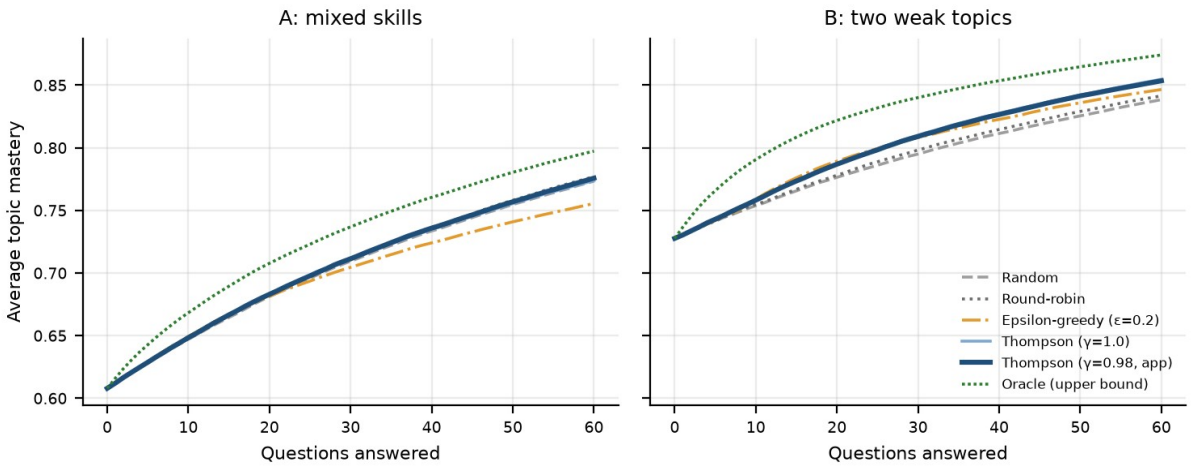


Figure 13. Average topic mastery during a 60-question session for each topic-selection policy.

For learners with two weak topics (scenario B), the Thompson-sampling bandit used in the app increased the learning gain by **13.6%** compared with random topic selection. It spent 41.2% of the questions on the two weakest topics (random: 24.9%) and raised the weakest topic's mastery from 0.617 to 0.688. The oracle, which knows the true weak topics, shows the ceiling (+32.3%). The bandit reaches about 42% of that improvement with no prior knowledge. Epsilon-greedy over-commits to one topic too early.

For learners whose skills are spread evenly (scenario A), no policy beat round-robin by much (the bandit gained 1.5% over random). When there is no clear weak spot, there is little to exploit. This is an honest limitation: the RL component is most useful for learners with specific blind spots. The undiscounted variant ($\gamma = 1.0$) did slightly better in scenario B and worse in scenario A, which motivated the compromise of $\gamma = 0.98$.

5.6 System Test Cases

Table 11. System test cases

TC	Test case and steps	Expected result	Actual result	Status
TC-01	Run `python -m pytest -q`	All unit tests pass	24 passed	Pass
TC-02	Run `python scripts/build_index.py`	Chunks and vectors built; kb_stats.json written	345 chunks; backends: lsa, ollama	Pass
TC-03	Start `streamlit run app.py` with Ollama running	Sidebar shows "LLM online: llama3.1:8b"	Shown in the Ask tab screenshot (Section 3.8)	Pass
TC-04	Ask "What does a flashing red traffic light mean?"	Answer says stop, then proceed when safe; cites FDM Vol. 1 p. 115	Correct, cited	Pass
TC-05	Ask an out-of-scope question (toll fee)	Refusal: not found in the manual	Refused	Pass
TC-06	Stop Ollama, then ask a question	Offline mode: quotes the best passage with its page; no crash	Extractive answer shown (Streamlit AppTest)	Pass
TC-07	Quiz tab: answer a question	Correct/incorrect feedback, explanation and source; bandit updated	Shown in screenshot	Pass
TC-08	Turn on "Let the LLM write new questions"	A valid generated question with its manual source	16/16 valid in evaluation	Pass

TC	Test case and steps	Expected result	Actual result	Status
TC-09	My Progress after several answers; click Reset	Mastery bars update; reset clears history	Shown in screenshot	Pass
TC-10	Run `python scripts/evaluate.py`	Reports written to reports/	All reports written	Pass

5.7 Summary of Objectives

Table 12. Objectives versus results

Objective	Target	Achieved	Status
O1 Knowledge base with citations	Both volumes, page-level	345 chunks, 246 pages	Met
O2 Retrieval	Recall@5 ≥ 0.85	0.96 (hybrid + nomic); BM25 0.85	Met
O3 Grounded answers	RAG > LLM-only; citations ≥ 90%; refusals	93.8% vs 56.2%; cited 95.8%; refusals 100.0%	Met
O4 RL adaptive quiz	Beats random in simulation	+13.6% (two weak topics); +1.5% (mixed)	Met (scenario B)
O5 Offline and reliable	Local models; fallback mode	Runs on localhost; extractive fallback tested	Met
O6 Code quality	Tests, README, comments	24 tests, CI, README, MIT licence	Met

6. Conclusion

6.1 Summary

DriveReady PH shows that a useful, trustworthy study assistant can be built entirely with open-source components on an ordinary PC. The official LTO manual was turned into 345 citable passages; a hybrid BM25 + embedding retriever finds the right page for most exam questions; Llama 3.1 8B, running locally through Ollama, answers from those passages with page citations and was 38 percentage points more accurate than the same model without the manual; and a Thompson-sampling bandit turns the quiz into an adaptive tutor that focuses on a learner's weak topics.

6.2 Key Takeaways and Lessons Learned

- **Grounding matters more than model size.** A mid-size local model with the right five passages is more reliable on country-specific facts than the same model alone, and citations let the user verify every answer.
- **Evaluate honestly, and watch for leakage.** The manual contains its own answer key. Holding it out was essential, because otherwise the scores would have measured lookup, not understanding.
- **Simple baselines are strong.** BM25, a method from the 1990s, was hard to beat on exam-style questions, and weak embeddings made hybrid search worse. Each component must be measured, not assumed to help.
- **RL helps when there is something to learn.** The bandit clearly helped learners with specific blind spots and gave little benefit when skills were evenly spread. That result is useful even though it is less exciting.
- **Engineering for failure pays off.** Health checks, an offline fallback, JSON validation, and a curated question bank mean the demo works even when the LLM misbehaves or is not running.
- **Course concepts were applied directly.** Tokenization and term statistics (Lesson 1) power BM25, and word vectors with cosine similarity (Lesson 4) power dense retrieval and the LSA fallback.

6.3 Future Work and Improvements

- Add the road-sign images through a vision-language model (e.g., LLaVA in Ollama) so that students can upload a photo of a sign and ask what it means.
- Add RA 4136 and the latest LTO memorandum circulars, and track the edition date of every passage.
- Run a real user study with license applicants to measure exam scores with and without the adaptive quiz.
- Use the thumbs-up/down feedback to re-rank passages (learning to rank) and to fine-tune the embedding model.
- Package the system as a mobile-friendly progressive web app, and offer a Filipino-language interface.

7. References

[1] Land Transportation Office, *Filipino Driver's Manual*, *Volume 1*, 2nd ed. Quezon City, Philippines: LTO Traffic Safety Division, 2021. [Online]. Available: https://lto.gov.ph/wp-content/uploads/2023/10/FDM-Vol.-1-2nd-Edition.pdf

[2] Land Transportation Office, *Filipino Driver's Manual*, *Volume 2*, 2nd ed. Quezon City, Philippines: LTO Traffic Safety Division, 2021. [Online]. Available: https://lto.gov.ph/wp-content/uploads/2023/10/FDM-vol.-2-2nd-Edition.pdf

[3] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[4] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[5] A. Grattafiori *et al.*, "The Llama 3 herd of models," 2024, *arXiv:2407.21783*.

[6] Ollama, "Ollama: Get up and running with large language models," 2026. [Online]. Available: https://ollama.com

[7] W. R. Thompson, "On the likelihood that one unknown probability exceeds another in view of the evidence of two samples," *Biometrika*, vol. 25, no. 3/4, pp. 285–294, 1933.

[8] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic Embed: Training a reproducible long context text embedder," 2024, *arXiv:2402.01613*.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[10] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759.

[11] O. Chapelle and L. Li, "An empirical evaluation of Thompson sampling," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 24, 2011, pp. 2249–2257.

[12] P. Bojanowski, E. Grave, A. Joulin, and T. Mikolov, "Enriching word vectors with subword information," *Trans. Assoc. Comput. Linguist.*, vol. 5, pp. 135–146, 2017.

[13] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.

[14] Y. Gao *et al.*, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.

[15] Streamlit Inc., "Streamlit documentation," 2026. [Online]. Available: https://docs.streamlit.io

[16] Republic of the Philippines, *Republic Act No. 4136: Land Transportation and Traffic Code*, Jun. 20, 1964.

8. Appendices

Appendix A. Repository Structure and How to Run

GitHub repository: **https://github.com/JEMJEM-0608/DriveReady-PH** (MIT licence). The complete instructions are in `README.md`.

Listing A1. Repository layout

```
DriveReady-PH/
├── app.py                      Streamlit web app (Ask / Quiz / My Progress / About)
├── driveready/
│   ├── config.py               all settings (paths, models, RAG and RL parameters)
│   ├── ingest.py               PDF -> clean text -> overlapping chunks with page numbers
│   ├── retriever.py            BM25, dense search, Reciprocal Rank Fusion
│   ├── knowledge.py            build/cache embeddings; choose Ollama (nomic) or LSA
│   ├── llm.py                  Ollama REST client (chat, embed, health)
│   ├── rag.py                  grounded prompt, citation validation, offline fallback
│   ├── quiz.py                 question bank + LLM question generation
│   ├── bandit.py               Thompson-sampling RL topic selection + baselines
│   └── storage.py              SQLite persistence
├── scripts/
│   ├── download_docs.py        fetch the LTO manuals
│   ├── build_index.py          build chunks + embeddings (writes reports/kb_stats.json)
│   ├── evaluate.py             retrieval, RAG vs LLM-only, quiz validity
│   └── simulate_bandit.py      RL experiment with simulated learners
├── data/  raw/ (PDFs) · processed/chunks.jsonl · eval/qa_eval.json · quiz/question_bank.json
├── reports/                    JSON + PNG results used in this document
├── tests/                      24 pytest unit tests
└── docs/                       documentation and figures
```

Listing A2. Commands to install, run, and evaluate

```
# quickest (Windows): setup_windows.bat, then run_app.bat / run_evaluation.bat
# manual setup (Windows PowerShell)
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
ollama pull llama3.1:8b
ollama pull nomic-embed-text
python scripts/download_docs.py      # or copy the two PDFs into data\raw\
python scripts/build_index.py

# run
streamlit run app.py                 # http://localhost:8501

# reproduce the results
python -m pytest -q
python scripts/simulate_bandit.py
python scripts/evaluate.py
```

Appendix B. Code Listings

The two modules that implement the core contributions (RAG and RL) are listed in full. The other modules are in the repository.

Listing B1. driveready/rag.py

```
"""Retrieval-Augmented Generation: question -> passages -> grounded, cited answer.

Pipeline
  1. Retrieve the top-k passages from the driver's manual (hybrid search).
  2. Put them, numbered [1]..[k], into a prompt with strict rules.
  3. The local LLM answers ONLY from those passages and cites them.
  4. A validator keeps only citation numbers that really exist and flags
     answers that cite nothing.
If Ollama is not running, the system still answers with the best passages
(extractive mode), so the app never breaks during a demo.
"""

from __future__ import annotations

import re
import time
from dataclasses import dataclass, field

from . import config
from .llm import OllamaClient, OllamaError
from .retriever import Hit, HybridRetriever

NOT_FOUND = "I could not find this in the LTO driver's manual."

SYSTEM_PROMPT = f"""You are DriveReady PH, a friendly reviewer for the Philippine LTO driver's
license exam.
Rules:
1. Answer ONLY with facts from the numbered manual passages provided. Never use outside
knowledge.
2. After each fact, cite its passage number in square brackets, e.g. [2].
3. If the passages do not contain the answer, reply exactly: "{NOT_FOUND}"
4. Keep the answer short: 2 to 5 sentences, in simple language.
   Reply in the language named in the "Answer language" line of the user message.
5. Do not invent fines, speeds, or law numbers that are not written in the passages."""


@dataclass
class Answer:
    question: str
    text: str
    hits: list[Hit]
    cited: list[int] = field(default_factory=list)   # 1-based passage numbers actually cited
    mode: str = "llm"                                 # "llm" or "extractive"
    grounded: bool = True                             # False if the LLM cited nothing
    timings: dict = field(default_factory=dict)       # retrieval_s, llm_s, total_s
    model: str = ""

    def sources(self) -> list[Hit]:
        """Passages to show under the answer (cited ones first)."""
        if self.cited:
            return [self.hits[i - 1] for i in self.cited]
        return self.hits


def format_context(hits: list[Hit]) -> str:
```

```
    blocks = []
    for i, h in enumerate(hits, start=1):
        c = h.chunk
        head = f"[{i}] ({c.doc_title}, page {c.page}{', ' + c.section if c.section else ''})"
        blocks.append(f"{head}\n{c.text}")
    return "\n\n".join(blocks)


_FILIPINO_WORDS = {"ang", "ng", "mga", "sa", "ano", "paano", "bakit", "kailan", "saan", "ba",
"po", "ko",
                   "ako", "mo", "ka", "kung", "dapat", "pwede", "puwede", "hindi", "may",
"nang", "lang"}


def detect_language(text: str) -> str:
    """Very small heuristic: 'Filipino' if the question uses common Tagalog function words.

    Llama 3.1 tends to switch to Filipino when it sees Philippine context, so the
    reply language is stated explicitly in every prompt (English by default)."""
    words = re.findall(r"[a-z]+", text.lower())
    hits = sum(w in _FILIPINO_WORDS for w in words)
    return "Filipino" if words and hits >= max(2, len(words) // 6) else "English"


def build_messages(question: str, hits: list[Hit], history: list[dict] | None = None) ->
list[dict]:
    msgs = [{"role": "system", "content": SYSTEM_PROMPT}]
    for turn in (history or [])[-4:]:          # short memory for follow-up questions
        msgs.append({"role": turn["role"], "content": turn["content"]})
    msgs.append({"role": "user", "content":
                 f"Manual passages:\n\n{format_context(hits)}\n\n"
                 f"Answer language: {detect_language(question)}\nQuestion: {question}"})
    return msgs


_CITE = re.compile(r"\[(\d+(?:\s*,\s*\d+)*)\]")


def extract_citations(text: str, n_passages: int) -> list[int]:
    """Return the valid, unique passage numbers cited in `text`, in order."""
    found: list[int] = []
    for group in _CITE.findall(text):
        for num in group.split(","):
            k = int(num)
            if 1 <= k <= n_passages and k not in found:
                found.append(k)
    return found


def strip_invalid_citations(text: str, n_passages: int) -> str:
    """Remove citation numbers that do not match any passage (e.g. [9] with 5 passages)."""
    def fix(m: re.Match) -> str:
        keep = [n.strip() for n in m.group(1).split(",") if 1 <= int(n) <= n_passages]
        return f"[{', '.join(keep)}]" if keep else ""
    return _CITE.sub(fix, text)
```

```
def extractive_answer(question: str, hits: list[Hit]) -> str:
    """Fallback when no LLM is available: quote the best passage."""
    if not hits:
        return NOT_FOUND
    best = hits[0].chunk
    sentences = re.split(r"(?<=[.!?])\s+", best.text)
    q_words = set(re.findall(r"[a-z0-9]+", question.lower()))
    ranked = sorted(sentences, key=lambda s: -len(q_words & set(re.findall(r"[a-z0-9]+",
s.lower()))))
    snippet = " ".join(ranked[:2]).strip()
    return f"(Offline mode - the LLM is not running.) Most relevant passage: \"{snippet}\"
[1]"


class RAGPipeline:
    def __init__(self, retriever: HybridRetriever, client: OllamaClient | None = None,
                 top_k: int = config.TOP_K):
        self.retriever = retriever
        self.client = client or OllamaClient()
        self.top_k = top_k

    def ask(self, question: str, history: list[dict] | None = None,
            mode: str = "hybrid", use_llm: bool = True) -> Answer:
        t0 = time.perf_counter()
        hits = self.retriever.search(question, k=self.top_k, mode=mode)
        t_ret = time.perf_counter() - t0
        if not use_llm or not self.client.available():
            text = extractive_answer(question, hits)
            return Answer(question, text, hits, cited=[1] if hits else [], mode="extractive",
                          timings={"retrieval_s": t_ret, "llm_s": 0.0,
                                   "total_s": time.perf_counter() - t0})
        try:
            res = self.client.chat(build_messages(question, hits, history))
        except OllamaError as e:
            text = extractive_answer(question, hits) + f"\n\n_LLM error: {e}_"
            return Answer(question, text, hits, cited=[1] if hits else [], mode="extractive",
                          timings={"retrieval_s": t_ret, "llm_s": 0.0,
                                   "total_s": time.perf_counter() - t0})
        text = strip_invalid_citations(res.text.strip(), len(hits))
        cited = extract_citations(text, len(hits))
        grounded = bool(cited) or NOT_FOUND.lower()[:30] in text.lower()
        return Answer(question, text, hits, cited=cited, mode="llm", grounded=grounded,
                      timings={"retrieval_s": t_ret, "llm_s": res.seconds,
                               "total_s": time.perf_counter() - t0},
                      model=res.model)
```

Listing B2. driveready/bandit.py

```
"""Adaptive quiz topic selection with reinforcement learning (multi-armed bandit).

The problem
  The reviewer has 8 quiz topics. A good tutor asks more questions about the
  topics a student keeps getting wrong, but still checks the other topics now
  and then. This is the classic exploration-vs-exploitation trade-off, which
  is exactly what a multi-armed bandit solves.

The RL formulation
  * Agent   : the quiz engine.
  * Actions : choose one of the K topics ("arms") for the next question.
  * Reward  : 1 if the student answers WRONG (we found a weak spot worth
              practising), 0 if correct.
  * Policy  : Thompson sampling. Each topic i keeps a Beta(alpha_i, beta_i)
              belief about its error rate. For every question we sample one
              value from each belief and pick the topic with the largest
              sample. Uncertain topics sometimes win (exploration); topics with
              many recorded mistakes usually win (exploitation).
  * Discount: students improve, so old mistakes should matter less. After
              every update all counts are multiplied by gamma (0.98) and pulled
              back toward the prior ("discounted Thompson sampling"), which lets
              the policy follow a learner whose skills change over time.
"""

from __future__ import annotations

from dataclasses import dataclass, field

import numpy as np

from . import config


@dataclass
class TopicBandit:
    topics: list[str] = field(default_factory=lambda: list(config.QUIZ_TOPICS))
    gamma: float = 0.98                       # discount factor (1.0 = never forget)
    prior: tuple[float, float] = config.BANDIT_PRIOR
    seed: int | None = None
    alpha: np.ndarray = field(init=False)     # "wrong" evidence per topic
    beta: np.ndarray = field(init=False)      # "correct" evidence per topic
    pulls: np.ndarray = field(init=False)     # how many questions per topic

    def __post_init__(self):
        k = len(self.topics)
        self.alpha = np.full(k, self.prior[0], dtype=float)
        self.beta = np.full(k, self.prior[1], dtype=float)
        self.pulls = np.zeros(k, dtype=int)
        self.rng = np.random.default_rng(self.seed)

    # -- policy ---------------------------------------------------------------
    def select(self) -> int:
        """Thompson sampling: sample each topic's error rate and pick the max."""
        samples = self.rng.beta(self.alpha, self.beta)
        return int(np.argmax(samples))
```

```
    def select_topic(self) -> str:
        return self.topics[self.select()]

    # -- learning ------------------------------------------------------------
    def update(self, arm: int, correct: bool) -> None:
        """Record one answer. reward = 1 for a wrong answer (weak spot found)."""
        a0, b0 = self.prior
        # discount: shrink all evidence toward the prior so recent answers count more
        self.alpha = a0 + self.gamma * (self.alpha - a0)
        self.beta = b0 + self.gamma * (self.beta - b0)
        if correct:
            self.beta[arm] += 1.0
        else:
            self.alpha[arm] += 1.0
        self.pulls[arm] += 1

    def update_topic(self, topic: str, correct: bool) -> None:
        self.update(self.topics.index(topic), correct)

    # -- reporting -----------------------------------------------------------
    def error_rate(self) -> np.ndarray:
        """Posterior mean error rate per topic."""
        return self.alpha / (self.alpha + self.beta)

    def mastery(self) -> dict[str, float]:
        """Estimated probability of answering correctly, per topic (0..1)."""
        return {t: float(1 - e) for t, e in zip(self.topics, self.error_rate())}

    # -- persistence ---------------------------------------------------------
    def to_dict(self) -> dict:
        return {"topics": self.topics, "alpha": self.alpha.tolist(), "beta":
self.beta.tolist(),
                "pulls": self.pulls.tolist(), "gamma": self.gamma}

    @classmethod
    def from_dict(cls, d: dict, seed: int | None = None) -> "TopicBandit":
        b = cls(topics=list(d["topics"]), gamma=d.get("gamma", 0.98), seed=seed)
        b.alpha = np.asarray(d["alpha"], dtype=float)
        b.beta = np.asarray(d["beta"], dtype=float)
        b.pulls = np.asarray(d["pulls"], dtype=int)
        return b


class OraclePolicy:
    """Upper bound used only in simulation: always knows the truly weakest topic."""

    def __init__(self, true_p: np.ndarray):
        self.p = true_p            # reference to the simulated learner's real skills

    def select(self) -> int:
        return int(np.argmin(self.p))

    def update(self, arm: int, correct: bool) -> None:
        pass
```

```
class RandomPolicy:
    """Baseline: choose a topic uniformly at random (no learning)."""

    def __init__(self, k: int, seed: int | None = None):
        self.k = k
        self.rng = np.random.default_rng(seed)

    def select(self) -> int:
        return int(self.rng.integers(self.k))

    def update(self, arm: int, correct: bool) -> None:
        pass


class RoundRobinPolicy:
    """Baseline: cycle through the topics in order (what a fixed reviewer does)."""

    def __init__(self, k: int):
        self.k, self.t = k, 0

    def select(self) -> int:
        arm = self.t % self.k
        self.t += 1
        return arm

    def update(self, arm: int, correct: bool) -> None:
        pass


class EpsilonGreedyPolicy:
    """Baseline RL policy: pick the topic with the highest observed error rate,
    but explore a random topic with probability epsilon."""

    def __init__(self, k: int, epsilon: float = 0.2, seed: int | None = None):
        self.k, self.eps = k, epsilon
        self.wrong = np.zeros(k)
        self.n = np.zeros(k)
        self.rng = np.random.default_rng(seed)

    def select(self) -> int:
        if self.rng.random() < self.eps or self.n.min() == 0:
            untried = np.flatnonzero(self.n == 0)
            return int(self.rng.choice(untried)) if len(untried) else
int(self.rng.integers(self.k))
        return int(np.argmax(self.wrong / self.n))

    def update(self, arm: int, correct: bool) -> None:
        self.n[arm] += 1
        self.wrong[arm] += 0 if correct else 1
```

Appendix C. Data Samples

Listing C1. One evaluation item (data/eval/qa_eval.json)

```
{
  "id": "Q26",
  "topic": "Parking & stopping",
  "question": "How far from a fire hydrant is parking prohibited?",
  "answerable": true,
  "gold": [
    {
      "doc": "FDM-Vol1",
      "page": 130
    },
    {
      "doc": "FDM-Vol2",
      "page": 79
    },
    {
      "doc": "FDM-Vol1",
      "page": 70
    }
  ],
  "must_include": [
    "4|four"
  ],
  "from": "Vol1 reviewer p184 #1"
}
```

Listing C2. One question-bank item (data/quiz/question_bank.json)

```
{
  "qid": "B001",
  "topic": "Road signs & traffic lights",
  "question": "What does a flashing red light mean?",
  "choices": [
    "Turn right only",
    "Slow down and proceed with caution",
    "Stop, then go only when it is safe to do so",
    "Proceed; you have the right of way"
  ],
  "answer": 2,
  "explanation": "A flashing red signal means bring your vehicle to a stop and proceed only
when it is safe.",
  "source": "Filipino Driver's Manual, Volume 1 (2nd Edition), p. 174 (Reviewer, official
answer)",
  "origin": "bank"
}
```

Table 13. Sample answers from the evaluation (reports/eval_answers.json)

ID	DriveReady (RAG) answer	LLM-only answer
Q01 ✓ / ✗	A flashing red traffic light means you must bring your vehicle to a STOP and proceed only when it is safe. [1]	A flashing red traffic light means drivers must come to a complete stop and yield to pedestrians and other vehicles. It is a warning to slow down and prepare to stop.

ID	DriveReady (RAG) answer	LLM-only answer
Q02 ✓ / ✓	When the traffic light is flashing yellow, a driver should slow down and proceed with caution. [1]	When the traffic light is flashing yellow, a driver should prepare to stop, but if it is safe to do so, they can proceed with caution. However, if the light is flashing yellow and red, they should come to a complete stop.
Q03 ✓ / ✓	A steady yellow or amber traffic signal indicates that the red signal is about to appear. [1]	A steady yellow or amber traffic signal indicates caution, warning drivers to prepare to stop or slow down before entering the intersection.
Q04 ✓ / ✓	These signs are called Regulatory Signs. [1]	Those are called "Traffic Signs" or more specifically, "Regulatory Signs". They inform road users of traffic laws and regulations, and disregarding them can result in penalties.
Q05 ✓ / ✓	When approaching a GIVE WAY sign at an intersection, you must give the right-of-way to any other vehicle in or closely approaching the intersection. [1] At a give-way sign intersection, you must give the right-of-way to any other vehicle in or closely approaching the intersection. [1] You must give way to pedestrians at crossi …	When approaching a GIVE WAY sign, a driver must slow down and give way to other traffic, pedestrians, or vehicles already on the road. They must yield the right of way to others and proceed only when it is safe to do so.
Q06 ✓ / ✓	A green signal means you can proceed, provided you yield to pedestrians and traffic lawfully using the intersection. [1] You must also yield to other vehicles and pedestrians lawfully within the intersection. [5] Proceed only after stopping and yielding to pedestrians and to cross-traffic vehicles if you are turning right on r …	A green traffic signal means that a driver can proceed with caution and pass through the intersection. It indicates that the traffic has been cleared and it is safe to drive through.

Appendix D. Evaluation Questions

ID	Topic	Question	Gold page(s)	Required facts	RAG	LLM
Q01	Road signs & traffic lights	What does a flashing red traffic light mean?	Vol1 p.115	stop; safe	✓	✗
Q02	Road signs & traffic lights	What should a driver do when the traffic light is flashing yellow?	Vol1 p.115	caution	✓	✓
Q03	Road signs & traffic lights	What does a steady yellow or amber traffic signal indicate?	Vol1 p.115	red|stop	✓	✓
Q04	Road signs & traffic lights	What do you call signs that inform road users of traffic laws, which constitute an offense if disregarded?	Vol1 p.66	regulatory	✓	✓
Q05	Road signs & traffic lights	What must a driver do when approaching a GIVE WAY sign?	Vol1 p.66, Vol1 p.153	yield|give way|give the right	✓	✓
Q06	Road signs & traffic lights	What does a green traffic signal mean for a driver?	Vol1 p.115	proceed	✓	✓
Q07	Pavement markings	What do double solid yellow lines on the road mean?	Vol1 p.95	overtak; cross	✓	✗
Q08	Pavement markings	What is allowed and not allowed on a single solid yellow line?	Vol1 p.95	overtak	✓	✗
Q09	Pavement markings	What does a broken yellow line allow motorists to do?	Vol1 p.96	overtak|pass	✓	✓
Q10	Pavement markings	What does a yellow box marking inside an intersection mean?	Vol1 p.100	stay|stop|block	✓	✓

ID	Topic	Question	Gold page(s)	Required facts	RAG	LLM
Q11	Pavement markings	What are rumble strips on the road for?	Vol1 p.102	warn|alert	✓	✓
Q12	Right of way & intersections	If two vehicles approach an intersection at about the same time, which one has the right of way?	Vol1 p.120, Vol1 p.153	on the right	✓	✓
Q13	Right of way & intersections	Who has the right of way at a roundabout or rotunda?	Vol1 p.126, Vol1 p.98	inside|within|already	✓	✓
Q14	Right of way & intersections	To which vehicles must a driver on a highway yield the right of way?	Vol1 p.121, Vol1 p.147	ambulance|police|fire	✓	✓
Q15	Right of way & intersections	What should a driver do when a pedestrian is crossing within a crosswalk?	Vol1 p.120	pedestrian; yield|stop	✓	✓
Q16	Right of way & intersections	What must a driver entering a highway from a private road or driveway do?	Vol1 p.121	yield	✓	✓
Q17	Right of way & intersections	What must a driver do before crossing a railroad crossing or a through highway?	Vol1 p.121	stop	✓	✓
Q18	Speed, overtaking & lanes	What is the maximum speed when passing through a school zone?	Vol1 p.118	20	✗	✗
Q19	Speed, overtaking & lanes	What is the maximum speed limit on expressways?	Vol1 p.132	100	✓	✓
Q20	Speed, overtaking & lanes	What is the minimum speed on an expressway?	Vol1 p.132	60	✓	✓
Q21	Speed, overtaking & lanes	What is the speed limit for passenger cars on open country roads with no blind corners?	Vol1 p.118	80	✓	✗
Q22	Speed, overtaking & lanes	What is the speed limit for trucks and buses on open country roads?	Vol1 p.118	50	✓	✗
Q23	Speed, overtaking & lanes	What is Lane 1 (the innermost lane) of an expressway used for?	Vol1 p.132	overtak|passing	✓	✓
Q24	Speed, overtaking & lanes	Which motorcycles are not allowed to enter the expressway?	Vol1 p.131	400	✓	✓
Q25	Speed, overtaking & lanes	In what situations do the speed limits not apply?	Vol1 p.119	ambulance|physician|emergency	✗	✓
Q26	Parking & stopping	How far from a fire hydrant is parking prohibited?	Vol1 p.130, Vol2 p.79, Vol1 p.70	4|four	✓	✗
Q27	Parking & stopping	Within how many meters of an intersection is parking prohibited?	Vol1 p.130, Vol2 p.79	6|six	✓	✗
Q28	Parking & stopping	Within what distance of a STOP sign is parking not allowed?	Vol1 p.66	6|six	✓	✗
Q29	Parking & stopping	At what angle should a vehicle be parked where angle parking is required?	Vol1 p.129	45|forty	✓	✓
Q30	Licensing & documents	How long is a student-driver's permit valid?	Vol1 p.8	one (1) year|1 year|one year	✓	✗
Q31	Licensing & documents	What is the minimum age to apply for a student-driver's permit?	Vol1 p.15	16|sixteen	✓	✓
Q32	Licensing & documents	How old must you be to apply for a new non-professional driver's license?	Vol1 p.19	17|seventeen	✓	✗

ID	Topic	Question	Gold page(s)	Required facts	RAG	LLM
Q33	Licensing & documents	Who is entitled to a driver's license valid for ten years?	Vol1 p.19, Vol2 p.60	10|ten; violation	✓	✗
Q34	Licensing & documents	What language requirement must a driver's license applicant meet?	Vol1 p.15, Vol1 p.19	read	✓	✓
Q35	Traffic laws & penalties	What is the penalty for the third offense of not wearing a seat belt?	Vol2 p.38	5,000|5000; week	✓	✗
Q36	Traffic laws & penalties	What is the fine for drunk driving if no one was injured?	Vol2 p.49	20,000|20000; 80,000|80000	✓	✗
Q37	Traffic laws & penalties	What happens to the license of a professional driver convicted of drunk driving for the first time?	Vol2 p.49	perpetual| revocation| revoked	✓	✗
Q38	Traffic laws & penalties	What happens if a driver does not pay the penalty for a traffic violation within 15 days?	Vol2 p.76	suspen	✓	✓
Q39	Traffic laws & penalties	How long is a Temporary Operator's Permit (TOP) valid?	Vol2 p.76	72|seventy	✓	✗
Q40	Traffic laws & penalties	When may a driver use a mobile phone under the Anti-Distracted Driving Act?	Vol2 p.53	emergency	✓	✗
Q41	Traffic laws & penalties	What is the penalty for the third offense under the Children's Safety on Motorcycles Act (RA 10666)?	Vol2 p.51	10,000|10000; suspen	✓	✗
Q42	Traffic laws & penalties	Under RA 11229, up to what age is a person considered a child who needs a child restraint system?	Vol2 p.64	12|twelve	✓	✓
Q43	Traffic laws & penalties	What blood alcohol concentration is considered drunk for a private car driver?	Vol2 p.43	0.05	✗	✓
Q44	Traffic laws & penalties	What is the penalty for driving without a valid driver's license?	Vol2 p.76	3,000|3000	✓	✗
Q45	Safe driving & emergencies	What is the recommended following distance behind another vehicle?	Vol1 p.117	second	✓	✓
Q46	Safe driving & emergencies	What is the total stopping distance of a car running at 100 kph?	Vol1 p.116	80	✓	✗
Q47	Safe driving & emergencies	What should you do if your vehicle starts to skid?	Vol1 p.149	brake|steer	✓	✓
Q48	Safe driving & emergencies	How are hand signals for starting, stopping or turning given?	Vol1 p.126	left	✓	✗
U01	Out of scope	What is the toll fee from Manila to Baguio on NLEX and TPLEX?	—	(refuse)	✓	✓
U02	Out of scope	Which car brand has the best resale value in the Philippines?	—	(refuse)	✓	✓
U03	Out of scope	What time does the LTO office in Quezon City open on Saturdays?	—	(refuse)	✓	✓
U04	Out of scope	How do I replace the battery of a Tesla Model 3?	—	(refuse)	✓	✓
U05	Out of scope	What is the current price of a liter of diesel in Metro Manila?	—	(refuse)	✓	✓

Appendix E. Use of AI Tools

In keeping with the course policy on generative AI, the team discloses that Anthropic's Claude was used as an assistant to: (1) brainstorm and compare project topics; (2) generate and debug code for the ingestion,

retrieval, RAG, quiz, bandit, storage, and Streamlit modules and the unit tests; (3) draft diagrams and this document. The team chose the topic and scope, supplied the official LTO manuals, ran all installations, models, tests, and evaluations on its own computer, checked the reported numbers against the JSON reports in the repository, and is fully responsible for the content of this submission. The open-source LLM evaluated in the project (Llama 3.1 8B via Ollama) is separate from the AI assistant used during development.